Algorithms Assignment 5

1.

1) $T(n) = 5T(n/7) + 3n^2$

Step 1

Substitute $T(n/7) = 5T(n/49) + 3(n/7)^2$

$$T(n) = 5\left[5T(n/49) + 3(n/7)^2\right] + 3n^2$$

$$5^2 T(n/7^2) + 5 \cdot 3\frac{n^2}{7^2} + 3n^2$$

Step 2

Substitute $T(n/49) = 5T(n/343) + 3(n/49)^2$

$$T(n) = 5^3 T(n/7^3) + 5^2 \cdot 3\frac{n^2}{7^4} + 5 \cdot 3\left(\frac{n^2}{7^2}\right) + 3n^2$$

Pattern

$$T(n) = 5^k \cdot T\left(\frac{n}{7^k}\right) + 3n^2 \sum_{i=0}^{k-1} \left(\frac{5}{49}\right)^i$$

each substitution multiplies the recursive term by 5 and shrinks the argument by a factor of 7, and the summation of geometric series with ratio 5/49

Proof by repeated substitution

First, we establish a closed form for $T(n)$ by induction on the number of substitutions (k), then evaluate the result at base case

For all $k \geq 0$: $T(n) = 5^k \cdot T\left(\frac{n}{7^k}\right) + 3n^2 \sum_{i=0}^{k-1} \left(\frac{5}{49}\right)^i$

Base case ($k=0$) then

$5^0 \cdot T\left(\frac{n}{7}\right) + 0 = T(n)$ holds true

Inductive step

$$T(n) = 5^k \cdot T\left(\frac{n}{7^k}\right) + 3n^2 \sum_{i=0}^{k-1} \left(\frac{5}{49}\right)^i$$

by the recurrence

$$T\left(\frac{n}{7^k}\right) = 5T(n/7^{(k+1)}) + 3(n/7^k)^2$$

Substitute this

$$T(n) = 5^k\left[5T\left(\frac{n}{7^{k+1}}\right) + 3\left(\frac{n}{7^k}\right)^2\right] + 3n^2 \sum_{i=0}^{k-1} \left(\frac{5}{49}\right)^i$$

$$= 5^{k+1} T\left(\frac{n}{7^{k+1}}\right) + 3n^2\left(\frac{5^k}{7^{2k}}\right) + 3n^2 \sum_{i=0}^{k-1} \left(\frac{5}{49}\right)^i$$

$$= 5^{k+1} T\left(\frac{n}{7^{k+1}}\right) + 3n^2\left(\frac{5}{49}\right)^k + 3n^2 \sum_{i=0}^{k-1} \left(\frac{5}{49}\right)^i$$

$$= 5^{k+1} T\left(\frac{n}{7^{k+1}}\right) + 3n^2 \sum_{i=0}^{k} \left(\frac{5}{49}\right)^i$$ which is the formula at $k+1$ $\square$

Evaluate base case

Set $k = \log_7 n$, so that $\frac{n}{7^k} = 1$ and $T(n/7^k) = T(1)$

By identity, $5^{\log_7 n} = n^{\log_7 5}$ so recursive term is

$$\Theta(n^{\log_7 5}) = \Theta(n^{.827})$$

The sum is being multiplied by $3n^2$ which $2 > .827$ so n^2 is the dominating term in the base case function

Therefore, $T(n) = \Theta(n^2)$ $\square$

Master Theorem = $T(n) = aT(n/b) + f(n)$

Recurrence = $T(n) = 5T(n/7) + 3n^2$

$a = 5,\ b = 7,\ f(n) = 3n^2$

$\log_b a = \log_7 5 \approx .827$

Case 3 $f(n) > n^{\log_7 5}$

$f(n) = 3n^2 = \Theta(n^2)$

2.

2) a) $T(n) = 5T\left(\frac{n}{9}\right) + n^7$

$a = 5 \quad b = 9 \quad f(n) = n^7$

$\log_9 5 \approx .732$

$n^7 > n^{\log_9 5}$ Case 3 $T(n) = O(n^7)$

b) $T(n) = 3T\left(\frac{n}{3}\right) + 7n$

$a = 3 \quad b = 3 \quad f(n) = 7n$

$\log_3 3 = 1$

$f(n) = 7n = O(n) = n^{\log_3 3}$ Case 2 applies $T(n) = O(n \log n)$

c) $T(n) = 15T\left(\frac{n}{4}\right) + 100$

$a = 15 \quad b = 4 \quad f(n) = 100 = n^0$

$\log_4 15 \approx 1.953$

$O(n^0) < n^{\log_4 15}$ Case 1 applies $T(n) = O(n^{\log_4 15})$

d) $T(n) = 9T\left(\frac{n}{3}\right) + n^2$

$a = 9,\ b = 3 \quad f(n) = n^2$

$\log_3 9 = 2$

$f(n) = n^2 = n^{\log_3 9}$ Case 2 $T(n) = O(n^2 \log n)$

e) $T(n) = 6T\left(\frac{n}{5}\right) + n^3$

$a = 6 \quad b = 5 \quad f(n) = n^3$

$\log_5 6 \approx 1.113$

$n^3 > n^{\log_5 6}$ Case 3 $T(n) = O(n^3)$

3. We will handle strings of different lengths by left-padding with (_) symbol.
   All the padded strings are: ___BOY, ___RIS, ___GUA, ___ATL, ___SUC, ___CES, ___QUI, ___PUT, ___ANT, ___VAU, ___HUI, ___VAL, ___SAN, ___SAN, ___CAQ, ___VIC, ___CAL, LA_GUA, ___TOL, __META, ___CAU, ___BOL, ___AMA, ___MAG, ___CHO, ___NSA, ___GUA, ___CAS, ___COR

   Pass 1 – sort by the character at position 6 (rightmost character)
   GUA LA_GUA META AMA NSA GUA | SUC VIC | MAG | CHO | QUI HUI |
   ATL VAL CAL TOL BOL | SAN SAN | CAQ | COR | RIS CES CAS | PUT ANT | VAU CAU | BOY
   Pass 2 – sort by position 5
   MAG VAL CAL SAN SAN CAQ CAS VAU CAU | CES | CHO | VIC RIS | AMA | ANT | TOL BOL COR BOY | NSA | META ATL | GUA LA_GUA GUA SUC QUI HUI PUT
   Pass 3 – sort by position 4
   AMA ANT ATL | BOL BOY | CAL CAQ CAS CAU CES CHO COR | META | GUA LA_GUA GUA | HUI | MAG | NSA | PUT | QUI | RIS | SAN SAN SUC | TOL | VAL VAU VIC
   Pass 4 – the _ symbol will sort before any letters. Sort by position 3
   AMA ANT ATL BOL BOY CAL CAQ CAS CAU CES CHO COR GUA GUA HUI MAG NSA PUT QUI RIS SAN SAN SUC TOL VAL VAU VIC LA_GUA | META
   Pass 5 – sort by position 2
   AMA ANT ATL BOL BOY CAL CAQ CAS CAU CES CHO COR GUA GUA HUI MAG NSA PUT QUI RIS SAN SAN SUC TOL VAL VAU VIC META | LA_GUA
   Pass 6 – sort by position 1
   AMA ANT ATL BOL BOY CAL CAQ CAS CAU CES CHO COR GUA GUA HUI MAG META NSA PUT QUI RIS SAN SAN SUC TOL VAL VAU VIC | LA_GUA

4.

4) $m = 7$ $\quad h1(key) = (|key - 9| \times |key + 13| / 15 + key) \% m$

$h2(key) = (key^3 + 1) \% m$

key = 62 $\quad h1 = (53 \times 75 = 3975, \div 15 = 265, + 62 = 327) \bmod 7 = 5$ = home slot

No collision → place in slot 5

key = 62 $\quad$ already in slot 5 (duplicate) → skipped

key = 69 $\quad h1 = (60 \times 82 = 4920, \div 15 = 328, + 69 = 397) \bmod 7 = 5$ = home slot

Probe 5 = Collision, slot 5 is full

$h2 = (69^3 + 1 = 328510) \bmod 7 = 0$ step size

(home + i x step) mod m → Probing 5 → 5 → 5 → 5 → resize m = 17 (next prime after 14)

(5 + i x 0) mod m

(5 mod 7) = 5

key 62 = h1 = 327 mod 17 = 4 = home slot

key 69 = h1 = 397 mod 17 = 6 = home slot $\quad$ no collisions here

key 70 $\quad h1 = (61 \times 83 = 5063, \div 15 = 337, + 70 = 407) \bmod 17 = 16$ = home slot

Probing [16] $\quad$ collisions = 0

key 4 $\quad h1 = (5 \times 17 = 85, \div 15 = 5, + 4 = 9) \bmod 17 = 9$ = home slot

Probing [9] $\quad$ collisions = 0

key 103 $\quad h1 = (94 \times 116 = 10904, \div 15 = 726, + 103 = 829) \bmod 17 = 13$ = home slot

Probing [13] $\quad$ collisions = 0

key 85 $\quad h1 = (76 \times 98 = 7448, \div 15 = 496, + 85 = 581) \bmod 17 = 3$ = home slot

Probing [3] $\quad$ collisions = 0

key 27 $\quad h1 = (18 \times 40 = 720, \div 15 = 48, + 27 = 75) \bmod 17 = 7$ = home slot

Probing [7] $\quad$ collisions = 0

key 86 $\quad h1 = (77 \times 99 = 7623, \div 15 = 508, + 86 = 594) \bmod 17 = 16$ = home slot

Probe [16] = collision = 1, slot 16 is full

$h2 = (86^3 + 1 = 636057) \bmod 17 = 2$ step size $(16 + 1 \times 2 = 18) \bmod 17 = 1$

key 40 $\quad h1 = (31 \times 53 = 1643, \div 15 = 109, + 40 = 149) \bmod 17 = 13$ = home slot

Probing [13] = collision = 1

$h2 = (40^3 + 1 = 64001) \bmod 17 = 13$ step size $\quad 13 + 1 \times 13 = 26 \bmod 17 = 9$

Probing [9] = collision = 2 $\quad 13 + 2 \times 13 = 39 \bmod 17 = 5$

Probing [5], open

key 96 $\quad h1 = (87 \times 109 = 9483, \div 15 = 632, + 96 = 728) \bmod 17 = 14$ = home slot

Probing [14] $\quad$ collisions = 0

key 77 $\quad h1 = (68 \times 90 = 6120, \div 15 = 632, + 77 = 485) \bmod 17 = 9$ = home slot

probing [9] $\quad$ collision = 1

$h2 = (77^3 + 1 = 456534) \bmod 17 = 16 \quad 9 + 1 \times 16 = 25 \bmod 17 = 8$

probing [8], open

key 23 $\quad h1 = (14 \times 36 = 504, \div 15 = 33, + 23 = 56) \bmod 17 = 5$ = home slot

probing [5] $\quad$ collisions = 1

$h2 = (23^3 + 1 = 12168) \bmod 17 = 13 \quad 5 + 1 \times 13 = 18 \bmod 17 = 1$

probing [1] $\quad$ collisions = 2 $\quad 5 + 2 \times 13 = 31 \bmod 17 = 14$

probing [14] $\quad$ collisions = 3 $\quad 5 + 3 \times 13 = 44 \bmod 17 = 10$

probing [10], open

	85		86	62	40	69	27	77	4	23			103	96		70
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

final hash table

5. On github

6. On github
7. 5) Time complexity: O(n * m) where n is the number of strings in the given array and m is the length of the longest string in the input array. In radixSort, the first loop visits every string once to check its length, which is O(n), where n is the number of strings in the array. Then the second loop calls countSort m times, where is the length of the longest string, once per character position. In countSort, the first for loop always iterates 27 times which is O(1). The second loop runs n times to visit every string once and perform charAt (O(1)), simple math (O(1)), and HashMap.get() and add() (O(1)), which makes the loop total cost O(n)+O(1) = O(n). For the final nested for loop, the outer loop runs 27 times (O(1)) and the inner loop visits every string exactly once across all buckets, which means O(1)*O(n) = O(n). One call to countSort costs O(n). In total, radixSort is O(n*m) because it calls countSort (O(n)), m times where m is the length of the longest string because the count sort is called on every position of the longest string once.
   Space complexity: O(n) where n is the number of strings in the input array because you have to create an array to hold the results in the sorted order with the same size as the original input. RadixSort creates a constant space to create the maxLength variable. Countsort creates a hashmap with a constant of 27 empty arrays created, which is O(1) because we don't care about constants. Count sort creates the result array, which is the size of n.
   6) Time Complexity: The time complexity is O(n^2 * L^2) where n is the number of strings in the array and L is the length of the longest string. The for loop that normalizes every word to lowercase has a complexity of O(n*L) since it visits every character of every string once. The second for loop hashes the string, which reads every character O(L), n times, which makes it O(n*L). The triple nested loop, the outermost loop runs n times, the next loop runs n times, and the final inner loop runs up to 2L times (total length of combined). The outer loops give O(n^2) pairs. For each pair: String concatenation copies every character so it is O(L) per pair. The split loop runs up to 2L times. isPalindrome scans up to O(L) characters, reverse copies then reverses (O(L)) and hashing the candidate string reads every character (O(L)). Every operation inside the split loop runs O(L) and the loop itself runs O(L) so the time complexity of the loop is O(L^2) per pair. This makes the total time complexity O(n^2 * L^2), since for each pair of strings (n^2), the loops runs (L^2) times. There is a bounded input of n and L never exceeding 300, so there is a cap of 8.1 billion operations in the worst case due to these constraints.
   Space complexity: The space complexity is O(n*L + n^3), where n is the number of strings in the array and L is the length of the longest string. This is because the normalization loop creates a new array of n strings, up to the length L, which is

O(n*L). The results list stores triplet palindromes in a list, and in the worst case (every combination is valid) it could be O(n^3) triplets. Together it is a space complexity of O(n*L + n^3). Given the constraints, O(n^3) is the dominant term, so the overall space complexity is O(n^3).